SMART ECCD

Child Care Center Management Platform

Full-Stack Technical Blueprint

Node.js · Express.js · React.js · PostgreSQL

Powered by Bloom's Taxonomy

Version 1.0 · 2025

CONFIDENTIAL

1. Technology Stack Overview

BloomKids is built on a modern JavaScript-first stack optimised for developer productivity, scalability, and rich data visualisation.

Layer	Technology	Version	Purpose
Frontend Framework	React.js	18.x	Component-based SPA
Routing (Client)	React Router v6	6.x	Role-based routing
Styling	TailwindCSS	3.x	Child-themed UI
Charts & Graphs	Recharts	2.x	Dashboard visualisations
State Management	Zustand	4.x	Lightweight global state
HTTP Client	Axios	1.x	API communication
Backend Runtime	Node.js	20 LTS	Server runtime
Backend Framework	Express.js	4.x	REST API server
Authentication	JWT + bcryptjs	—	Secure token auth
RBAC	Custom middleware	—	Role-based access
ORM	Prisma	5.x	Type-safe DB queries
Database	PostgreSQL	16.x	Primary data store
File Storage	Multer + local/S3	—	Media uploads
Email	Nodemailer	6.x	Notifications & alerts
PDF Reports	Puppeteer	21.x	Report generation
Real-time	Socket.io	4.x	Live dashboard updates
Validation	Zod	3.x	Schema validation
Testing	Jest + Supertest	—	Unit & API testing
Build Tool	Vite	5.x	Frontend bundler
Deployment	Railway / Render	—	Cloud hosting

2. Project Folder Structure

The project uses a monorepo layout with clearly separated client and server workspaces.

2.1 Root Structure

```
bloomkids/
├── client/                 # React Frontend (Vite)
├── server/                 # Express + Node.js Backend
├── .env.example            # Environment variables template
├── docker-compose.yml      # Local dev (PostgreSQL)
├── package.json            # Root workspace config
└── README.md
```

2.2 Client (React) Structure

```
client/
├── public/                 # Static assets, favicon
├── src/
│   ├── assets/             # Images, icons, illustrations
│   ├── components/         # Reusable UI components
│   │   ├── common/         # Button, Card, Modal, Badge
│   │   ├── charts/         # BloomRadar, TrendLine, HeatMap
│   │   ├── layout/         # Navbar, Sidebar, Footer
│   │   └── forms/          # ActivityForm, PerformanceForm
│   ├── pages/              # Route-level page components
│   │   ├── auth/           # Login, ForgotPassword
│   │   ├── superadmin/     # SuperAdmin dashboard & pages
│   │   ├── manager/        # Center Manager pages
│   │   ├── teacher/        # Teacher dashboard & pages
│   │   └── parent/         # Parent dashboard & pages
│   ├── hooks/              # Custom React hooks
│   ├── context/            # AuthContext, ThemeContext
│   ├── services/           # Axios API service wrappers
│   ├── store/              # Zustand state stores
│   ├── utils/              # Helpers, formatters, constants
│   ├── styles/             # Global CSS, Tailwind config
│   ├── App.jsx             # Root component with routing
│   └── main.jsx            # React entry point
├── tailwind.config.js
├── vite.config.js
└── package.json
```

2.3 Server (Node/Express) Structure

```
server/
├── prisma/
│   ├── schema.prisma       # Full database schema
│   ├── migrations/         # Auto-generated migrations
│   └── seed.js             # Database seeding script
```

```
├── src/
│   ├── config/              # DB, email, storage config
│   ├── controllers/         # Request handlers
│   │   ├── auth.controller.js
│   │   ├── user.controller.js
│   │   ├── center.controller.js
│   │   ├── activity.controller.js
│   │   ├── performance.controller.js
│   │   ├── attendance.controller.js
│   │   ├── report.controller.js
│   │   └── message.controller.js
│   ├── middleware/
│   │   ├── auth.middleware.js      # JWT verification
│   │   ├── rbac.middleware.js      # Role permission guards
│   │   ├── upload.middleware.js    # Multer file handling
│   │   └── validate.middleware.js  # Zod request validation
│   ├── routes/              # Express route definitions
│   ├── services/            # Business logic layer
│   │   ├── bloom.service.js        # Bloom's analytics engine
│   │   ├── notification.service.js
│   │   ├── report.service.js
│   │   └── email.service.js
│   ├── socket/              # Socket.io event handlers
│   ├── utils/               # Helpers, constants
│   └── app.js               # Express app setup
├── server.js                # HTTP + Socket.io entry point
└── package.json
```

CONFIDE

3. Database Schema (Prisma / PostgreSQL)

3.1 Enums

```
enum Role {
  SUPER_ADMIN
  CENTER_MANAGER
  TEACHER
  PARENT
}

enum BloomLevel {
  REMEMBER
  UNDERSTAND
  APPLY
  ANALYZE
  EVALUATE
  CREATE
}

enum ActivityStatus {
  DRAFT | PUBLISHED | ARCHIVED
}

enum AssignmentStatus {
  PENDING | IN_PROGRESS | COMPLETED | SKIPPED
}

enum AttendanceStatus {
  PRESENT | ABSENT | LATE | EXCUSED
}

enum CompletionStatus {
  COMPLETED | PARTIAL | NOT_ATTEMPTED
}
```

3.2 Core Models

```
model User {
  id           String   @id @default(cuid())
  name         String
  email        String   @unique
  passwordHash String
  role         Role
  centerId     String?
  avatar       String?
  isActive     Boolean  @default(true)
  createdAt    DateTime @default(now())
  center       Center?  @relation(fields: [centerId], references: [id])
  // role-specific relations
  managedCenter Center? @relation('CenterManager')
  teacherClasses Class[]
```

```
  childParents  ChildParent[]
  sentMessages  Message[] @relation('Sender')
}

model Center {
  id        String   @id @default(cuid())
  name      String
  address   String
  managerId String   @unique
  manager   User     @relation('CenterManager', fields: [managerId])
  users     User[]
  classes   Class[]
  activities Activity[]
}

model Class {
  id        String   @id @default(cuid())
  name      String
  ageGroup  String   // e.g. '3-4 years'
  centerId  String
  teacherId String
  center    Center   @relation(fields: [centerId], references: [id])
  teacher   User     @relation(fields: [teacherId], references: [id])
  children  Child[]
  assignments ActivityAssignment[]
}

model Child {
  id           String    @id @default(cuid())
  firstName    String
  lastName     String
  dateOfBirth  DateTime
  photo        String?
  classId      String
  centerId     String
  medicalNotes String?
  class        Class     @relation(fields: [classId], references: [id])
  parents      ChildParent[]
  performance  ChildPerformance[]
  attendance   Attendance[]
}

model ChildParent {
  childId  String
  parentId String
  child    Child  @relation(fields: [childId], references: [id])
  parent   User   @relation(fields: [parentId], references: [id])
  @@id([childId, parentId])
}
```

3.3 Activity & Performance Models

```
model Activity {
  id             String         @id @default(cuid())
  title          String
  description    String
  instructions   String
  bloomLevels    BloomLevel[]   // Array of targeted levels
```

```
  activityType   String         // Individual|Group|Outdoor|Creative
  ageGroup       String
  durationMins   Int
  resources      String[]       // File URLs
  learningGoals  String[]
  status         ActivityStatus @default(DRAFT)
  centerId       String
  createdById    String
  center         Center         @relation(fields: [centerId], references: [id])
  assignments    ActivityAssignment[]
}

model ActivityAssignment {
  id            String           @id @default(cuid())
  activityId    String
  classId       String
  teacherId     String
  scheduledDate DateTime
  scheduledTime String
  isRecurring   Boolean          @default(false)
  recurringRule String?          // RRULE string
  status        AssignmentStatus @default(PENDING)
  activity      Activity         @relation(fields: [activityId], references: [id])
  class         Class            @relation(fields: [classId], references: [id])
  record        ActivityRecord?
}

model ActivityRecord {
  id            String   @id @default(cuid())
  assignmentId  String   @unique
  conductedDate DateTime
  notes         String?
  assignment    ActivityAssignment @relation(fields: [assignmentId], references: [id])
  performances  ChildPerformance[]
  attendances   Attendance[]
}

model ChildPerformance {
  id                  String           @id @default(cuid())
  recordId            String
  childId             String
  completionStatus    CompletionStatus
  bloomLevelAchieved  BloomLevel
  skillRatings        Json             // { goal: rating 1-5 }
  observationNotes    String?
  evidenceUrls        String[]
  isFlagged           Boolean          @default(false)
  flagReason          String?
  createdAt           DateTime         @default(now())
  record              ActivityRecord   @relation(fields: [recordId], references: [id])
  child               Child            @relation(fields: [childId], references: [id])
}

model Attendance {
  id        String           @id @default(cuid())
  childId   String
  recordId  String?
  date      DateTime
  status    AttendanceStatus
  note      String?
  child     Child            @relation(fields: [childId], references: [id])
}
```

4. REST API Route Structure

4.1 Authentication Routes

Method	Status	Endpoint	Description
POST	Public	/api/auth/login	Login — returns JWT token
POST	Public	/api/auth/refresh	Refresh access token
POST	Auth	/api/auth/logout	Invalidate token
POST	Public	/api/auth/forgot-passwor d	Send OTP via email
POST	Public	/api/auth/reset-passwor d	Reset password with OTP
GET	Auth	/api/auth/me	Get current user profile

4.2 User Management Routes

Method	Role	Endpoint	Description
GET	SA	/api/users	List all users (paginated)
POST	SA | CM	/api/users	Create new user
GET	Auth	/api/users/:id	Get user by ID
PUT	SA | CM	/api/users/:id	Update user details
DELETE	SA	/api/users/:id	Soft delete user
POST	SA | CM	/api/users/:id/activate	Activate/deactivate user

4.3 Activity Routes

Method	Role	Endpoint	Description
GET	Auth	/api/activities	List activities (filtered)
POST	SA | CM	/api/activities	Create new activity
GET	Auth	/api/activities/:id	Get activity details
PUT	SA | CM	/api/activities/:id	Update activity
DELETE	SA | CM	/api/activities/:id	Archive activity
POST	SA | CM	/api/activities/:id/assign	Assign activity to class/teacher
GET	Teacher	/api/activities/assignmen ts/my	My assigned activities

Method	Role	Endpoint	Description
POST	Teacher	/api/activities/assignments/:id/conduct	Submit activity record

4.4 Performance & Analytics Routes

Method	Role	Endpoint	Description
GET	Auth	/api/performance/child/:id	Child performance history
GET	Auth	/api/performance/child/:id/bloom	Bloom's radar data per child
GET	CM | SA	/api/performance/class/:id	Class performance summary
GET	CM | SA	/api/performance/center/bloom-coverage	Center-wide Bloom's coverage
GET	Auth	/api/performance/child/:id/trend	Performance trend over time
GET	CM | SA	/api/dashboard/center-stats	Center homepage statistics
GET	Teacher	/api/dashboard/teacher-today	Teacher today's activities
GET	Parent	/api/dashboard/parent/:childId	Parent child summary

4.5 Additional Routes

Method	Role	Endpoint	Description
POST	Teacher	/api/attendance	Mark attendance
GET	Auth	/api/attendance/child/:id	Child attendance record
GET	Auth	/api/messages	Get messages/inbox
POST	Auth	/api/messages	Send message
GET	CM | SA	/api/reports/child/:id	Generate child PDF report
GET	CM | SA	/api/reports/center	Generate center report
POST	CM | SA	/api/announcements	Create announcement
GET	Auth	/api/notifications	Get notifications

5. Frontend Pages & Components

5.1 Route Map

Role	Route	Page / Component
Public	/login	Login Page
Public	/forgot-password	Password Reset Page
Super Admin	/sa/dashboard	SA Dashboard — center-wide stats
Super Admin	/sa/centers	Manage All Centers
Super Admin	/sa/users	Global User Management
Manager	/manager/dashboard	Center Stats Homepage
Manager	/manager/activities	Activity Library
Manager	/manager/activities/new	Activity Designer (Bloom's form)
Manager	/manager/activities/:id/assign	Assignment Scheduler
Manager	/manager/classes	Class & Teacher Management
Manager	/manager/children	All Children Directory
Manager	/manager/reports	Report Generator
Teacher	/teacher/dashboard	Today's Activities + Stats
Teacher	/teacher/activities	My Assigned Activities
Teacher	/teacher/activities/:id/conduct	Conduct Activity Flow
Teacher	/teacher/children	My Class Children
Teacher	/teacher/attendance	Mark Attendance
Parent	/parent/dashboard	Child Performance Overview
Parent	/parent/child/:id/performance	Detailed Performance + Filters
Parent	/parent/child/:id/attendance	Attendance Calendar View
Parent	/parent/messages	Messages with Teacher/Manager
All	/profile	User Profile Settings
All	/notifications	Notifications Centre

5.2 Key Component Descriptions

BloomRadarChart

A spider/radar chart built with Recharts displaying a child's performance across all 6 Bloom's Taxonomy levels. Each axis represents one level (Remember → Create). The filled area shows the child's current achievement. A second overlay shows class average for comparison. Used prominently on both Parent and Manager dashboards.

ActivityDesigner Form

The core Manager tool for creating activities. Includes: title/description fields, a Bloom's level multi-selector (each level shown with its colour badge and keyword hints), age group picker, activity type selector, learning goals builder (add/remove rows), file uploader for resources, and duration slider. All validated client-side with Zod before submission.

ConductActivityFlow

A step-based wizard for teachers. Step 1: mark attendance for the session. Step 2: for each child — set completion status, Bloom's level achieved, rate each learning goal 1-5 stars, add observation notes, and optionally upload a photo/video. Step 3: review summary and submit. The wizard saves progress to localStorage so teachers can resume if interrupted.

ParentPerformanceDashboard

The parent home page. Features a Bloom's radar chart, an activity feed timeline, trend line chart for overall performance over time, a filter sidebar (date range, Bloom's level, activity type, teacher), and an attendance heatmap calendar. All charts are interactive with tooltips. A child selector in the header switches between siblings.

CONFIDENTIAL

6. Bloom's Taxonomy Engine (Core USP)

The Bloom's engine is the analytical core of BloomKids. It aggregates individual child performance records into actionable insights at child, class, and center level.

6.1 Bloom's Level Reference

Level	Colour	Action Verbs	Indicator (Child achieves...)
Remember	Red #E74C3C	Recall, Identify, Name	Can name or point to items from the activity
Understand	Orange #E67E22	Explain, Describe, Sort	Can explain what happened or retell a story
Apply	Yellow #F1C40F	Use, Demonstrate, Solve	Attempts to use skill independently
Analyze	Green #27AE60	Compare, Group, Examine	Spots differences or groups items correctly
Evaluate	Blue #2980B9	Justify, Choose, Assess	Makes a choice and can explain why
Create	Purple #8E44AD	Design, Build, Produce	Produces something original from the activity

6.2 bloom.service.js — Key Functions

```
// Get Bloom's radar data for a child
getChildBloomProfile(childId, dateRange) {
  // Aggregates all ChildPerformance records
  // Returns: { REMEMBER: 72, UNDERSTAND: 65, APPLY: 80, ... }
  // Scores = avg of (levelAchieved weight × completion %) × 100
}

// Get center-wide Bloom's coverage
getCenterBloomCoverage(centerId, month) {
  // Shows which levels are under-represented in activity design
  // Returns: { REMEMBER: 45%, UNDERSTAND: 30%, CREATE: 5%, ... }
  // Flags levels below 10% coverage as 'Under-utilised'
}

// Get class progression over time
getClassBloomTrend(classId, weeks) {
  // Returns weekly data points for Recharts LineChart
  // One series per Bloom's level
}

// Flag developmental concerns
analyseChildProgress(childId) {
  // Compares child to class average per level
  // Returns flagged levels where child is >20% below average
  // Used to auto-suggest teacher follow-up actions
}
```

6.3 Bloom's Visual Design System

Every Bloom's level has a consistent colour identity used throughout the application:

🔴 REMEMBER → #E74C3C (Red) → Badge, chart segment, activity tag
🟠 UNDERSTAND → #E67E22 (Orange) → Badge, chart segment, activity tag
🟡 APPLY → #F1C40F (Yellow) → Badge, chart segment, activity tag
🟢 ANALYZE → #27AE60 (Green) → Badge, chart segment, activity tag
🔵 EVALUATE → #2980B9 (Blue) → Badge, chart segment, activity tag
🟣 CREATE → #8E44AD (Purple) → Badge, chart segment, activity tag

This colour consistency helps teachers and managers instantly identify cognitive levels at a glance.

CONFIDENTIA

7. Authentication & Role-Based Access Control

7.1 JWT Authentication Flow

```
// Login Flow
POST /api/auth/login
  → Validate credentials (bcrypt compare)
  → Generate accessToken (15min expiry)
  → Generate refreshToken (7 days, stored in httpOnly cookie)
  → Return { accessToken, user: { id, name, role, centerId } }

// Protected Route Flow
Request → auth.middleware.js
  → Extract Bearer token from Authorization header
  → Verify JWT signature
  → Attach req.user = { id, role, centerId }
  → Pass to rbac.middleware.js → check role permission
  → Pass to controller

// Token Refresh
POST /api/auth/refresh
  → Read refreshToken from httpOnly cookie
  → Verify and issue new accessToken
```

7.2 RBAC Middleware

```
// rbac.middleware.js
const permissions = {
  SUPER_ADMIN:     ['*'],   // full access
  CENTER_MANAGER:  ['center:read', 'center:write', 'activity:write',
                    'user:write', 'report:read', 'children:read'],
  TEACHER:         ['activity:conduct', 'attendance:write',
                    'performance:write', 'children:read:own-class'],
  PARENT:          ['performance:read:own-child', 'attendance:read:own-child',
                    'message:read', 'message:write'],
};

// Usage in routes
router.get('/dashboard/center-stats',
  authenticate,
  authorize('center:read'),  // middleware checks permission
  DashboardController.getCenterStats
);
```

8. Development Phases & Sprint Plan

Phase	Weeks	Deliverables	Key Milestones
Phase 1	1 – 3	Project setup, Docker, PostgreSQL, Prisma schema, JWT auth, RBAC middleware, user CRUD, role-based routing in React	Auth working for all 4 roles
Phase 2	4 – 7	Activity designer, Bloom's level tagging, activity library, assignment scheduler, teacher assignment view	Manager can design & assign activities
Phase 3	8 – 11	Conduct activity wizard, child performance recording, evidence uploads, Bloom's service analytics engine	Teacher can conduct & record activities
Phase 4	12 – 15	All dashboards, Recharts integration (radar, line, bar, heatmap), parent filters, child profiles, attendance	All 4 dashboards live with charts
Phase 5	16 – 18	Messaging system, push notifications, announcements, Socket.io real-time updates, email notifications	Real-time features working
Phase 6	19 – 20	PDF report generation (Puppeteer), Excel export, report templates, child portfolio view	Reports downloadable
Phase 7	21 – 22	QA testing, UAT with center staff, bug fixes, performance optimisation, mobile responsiveness review	Beta-ready build
Phase 8	23 – 24	Deployment to Railway/Render, domain & SSL setup, staff training, go-live	Production launch

8.1 Recommended Team Composition

Role	Count	Responsibilities
Full-Stack Developer	2	React frontend + Express backend, API integration
Frontend Developer	1	UI components, TailwindCSS, Recharts, mobile responsiveness
Backend Developer	1	Prisma schema, complex queries, Bloom's analytics engine
UI/UX Designer	1	Child-themed design system, wireframes, prototypes
QA Engineer	1	Test cases, UAT coordination, bug tracking
Project Manager	1	Sprint planning, stakeholder communication

9. Deployment Architecture

9.1 Recommended Deployment Stack

Service	Provider	Cost/Month	Notes
Frontend (React)	Vercel	Free	Auto-deploy from GitHub
Backend (Node/Express)	Railway	~$5	Auto-deploy, env vars, logs
Database (PostgreSQL)	Supabase	Free / $25	Managed Postgres + backups
File Storage	Cloudinary	Free tier	Images & video uploads
Email	Resend.com	Free tier	3,000 emails/month free
SSL Certificate	Auto (Vercel)	Free	Automatic HTTPS
Domain	Namecheap	~$12/year	bloomkids.com or similar
Total		~$10–30/mo	Scales with usage

9.2 Environment Variables

```
# Server .env
DATABASE_URL=postgresql://user:pass@host:5432/bloomkids
JWT_SECRET=your-super-secret-key-here
JWT_REFRESH_SECRET=your-refresh-secret-here
JWT_EXPIRES_IN=15m
JWT_REFRESH_EXPIRES_IN=7d
CLIENT_URL=https://bloomkids.vercel.app
CLOUDINARY_CLOUD_NAME=your-cloud-name
CLOUDINARY_API_KEY=your-api-key
CLOUDINARY_API_SECRET=your-api-secret
RESEND_API_KEY=re_xxxxxxxxxxxx
PORT=5000
```

9.3 CI/CD Pipeline

- Developer pushes to GitHub feature branch
- Pull request triggers automated Jest test suite
- On merge to main: Vercel auto-deploys frontend
- On merge to main: Railway auto-deploys backend
- Database migrations run via prisma migrate deploy on deploy
- Slack/email notification on successful deployment

10. Additional Features & Future Roadmap

10.1 Security Considerations

- Rate limiting on all auth endpoints (express-rate-limit)
- Helmet.js for HTTP security headers
- CORS configured to allow only the frontend domain
- All file uploads validated for type and size (images/videos only)
- SQL injection prevention via Prisma parameterised queries
- GDPR/COPPA compliant — parent consent on signup, data export/delete endpoints
- Sensitive fields (medical notes) encrypted at rest

10.2 Performance Optimisations

- Pagination on all list endpoints (cursor-based for large datasets)
- React.lazy and Suspense for code splitting per role dashboard
- Chart data cached in Zustand store with 5-minute TTL
- Database indexes on frequently queried columns (childId, centerId, scheduledDate)
- Prisma query optimisation with select (no over-fetching)

10.3 Mobile Experience

- TailwindCSS responsive breakpoints throughout
- PWA-enabled (service worker) for teacher offline activity recording
- Touch-friendly activity conduct wizard on mobile
- Parent dashboard optimised for phone-first usage

10.4 Phase 2 Roadmap Features

- Multi-language support (i18next) — English, Bahasa, Tamil, Mandarin
- AI-powered activity recommendations based on child's Bloom's gaps
- Parent mobile app (React Native — reuse component logic)
- Video call integration for parent-teacher meetings (Daily.co)
- Milestone tracker aligned to WHO early childhood development standards
- Multi-center franchise support under one Super Admin account
- Custom branded white-label version for enterprise centers

CONFIDENTIAL